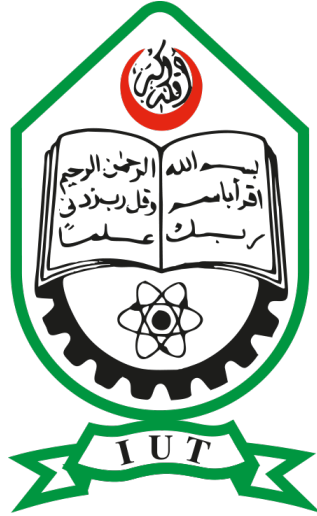




SPL-II Project Proposal: Vero

A Productivity Software Solution

Supervisor:

Anika Farzana

Junior Lecturer,

Department of Computer Science and Engineering,
Islamic University of Technology (IUT)

Team Members:

Nafis Ahnaf Jamil	ID: 230042150
Tabib Hassan	ID: 230042131
Sieam Shahriare	ID: 230042153

May 10, 2026

Contents

1	Project Overview	2
2	Motivation	2
3	The Case for Unified Workspaces: Past Research Findings	3
4	Requirement Analysis	4
4.1	Stakeholder Analysis	4
4.2	Real-life Stakeholder Feedback	4
4.3	Functional Requirements	5
4.4	Non-Functional Requirements	6
5	System Design	8
5.1	Client-Server Architecture	8
5.2	User Flow Chart	9
5.3	Workspace Separation	10
6	Key Features	10
6.1	Workspace & Organization	10
6.2	Meet Module (Communication)	11
6.3	Notes Module (Documentation)	11
6.4	Board Module (Task Management)	11
6.5	AI & Automation (Integrations)	11
7	Tools and Technology	11
7.1	Frontend Application:	12
7.2	Backend & Real-time Infrastructure:	12
7.3	External APIs & Services:	13
7.4	DevOps & Observability:	13
8	Proposed Timeline	14
9	Action Taken Based on Feedback	14

1 Project Overview

Vero is an integrated productivity ecosystem designed to eliminate the friction between communication and documentation. Conceived as a "fusion" of high-performance video conferencing and comprehensive workspace management, Vero provides a single application environment that replaces the need for separate, disconnected tools.

The platform is built around a **Shared Project Context** that tightly couples three core modules:

- **Meet Module:** A P2P audio and video conferencing suite featuring screen sharing and automated meeting note initialization.
- **Notes Module:** A collaborative rich text editor that utilizes slash commands and simultaneous note-taking capabilities.
- **Board Module:** A workflow planning tool featuring Kanban boards where tasks are dynamically linked to meeting notes and real-time checklists. Subsequently, people can be assigned tasks in a team and maintained by a manager.

2 Motivation

The primary motivation behind Vero is to address the fragmented nature of modern remote team work and academic collaboration. Currently, teams are forced to juggle at least two separate applications for video conferencing, documentation, and task assignment. This fragmentation leads to several critical pain points:

1. **Context Switching:** Valuable time and focus are lost when users must manually transfer information from a meeting (*Zoom/Teams*) to a document (*Notion/Docs*) and then to a task manager (*Trello/Jira*).
2. **Information Silos:** Action items discussed during live calls often fail to be documented, leading to "meeting amnesia" where decisions are lost or forgotten.
3. **Lack of Synchronization:** Changes in a project's task list (Board) are rarely reflected automatically in the related documentation, causing a disconnect between planning and execution.

3 The Case for Unified Workspaces: Past Research Findings

While modern collaborative tools like Zoom and Notion are powerful, the necessity of switching between them creates a significant "Tax on Productivity." Vero is designed specifically to mitigate these scientifically documented cognitive costs.

- **Rubinstein, Meyer & Evans (2001) – The Switch Cost**

Rubinstein, Meyer, and Evans at the University of Michigan ran four controlled experiments in which participants alternated between tasks of varying complexity. For all types of tasks, subjects lost time when switching, and time costs increased with complexity. Their model identifies two cognitive stages behind every switch: "goal shifting" (deciding to do one thing instead of another) and "rule activation" (turning off the rules for the old task and turning on rules for the new one), both consuming measurable time and cognitive resources.

Rubinstein, J. S., Meyer, D. E., & Evans, J. E. (2001). Executive control of cognitive processes in task switching. *Journal of Experimental Psychology: Human Perception and Performance*, 27(4), 763–797. <https://doi.org/10.1037/0096-1523.27.4.763>

- **Mark, Gudith & Klocke (2008) – Interruptions Cause Stress Even When You Catch Up**

This empirical study found that while interrupted workers completed tasks in less time with no quality difference, they compensated by working faster – at the cost of significantly higher stress, frustration, time pressure, and effort. After just 20 minutes of interrupted performance, participants reported measurably elevated stress and workload. This is directly applicable to remote team workflows that involve frequent switching between video conferencing, note-taking, and task management tools.

Mark, G., Gudith, D., & Klocke, U. (2008). The cost of interrupted work: More speed and stress. *Proceedings of the SIGCHI Conference on Human Factors in Computing Systems (CHI '08)*, 107–110. <https://doi.org/10.1145/1357054.1357072>

4 Requirement Analysis

4.1 Stakeholder Analysis

To better understand the ecosystem of **Vero**, we have identified three tiers of stakeholders:

Category	Stakeholder	Role
Primary	University Students	Use the platform for attending study sessions and real-time collaborative note-taking.
	Project Managers	Utilize the Kanban board and meet modules to oversee team workflows and deadlines.
	Software Engineers	Rely on P2P video and rich-text slash commands for technical documentation and sprint planning.
Secondary	Course Instructors	Review project progress and documentation through the shared workspace context.
	Workspace Admins	Manage user roles, permissions, and global dashboard settings for the organization.
External	Cloud Providers	Host the infrastructure for WebRTC, Gemini API, and CRDT synchronization data.
	Open Source Devs	Maintain and update the underlying libraries used for the block editor and UI components.

4.2 Real-life Stakeholder Feedback

For our requirement gathering, we interviewed the following stakeholders to understand their pain points and expectations from a unified productivity platform.

- **Mohammad Faiaj Jarif – 2nd Year Student, Islamic University of Technology (IUT)**

”During group projects, we constantly switch between Messenger for calls, Google Docs for notes, and Trello for tasks. It’s exhausting to keep everything in sync. I wish there was one place where our meeting notes automatically linked to our task board.”

- **Faiyaz Awsaf – CEO, AFK Tech Limited (Software Startup)**

”As a startup founder, I need visibility into what my team is working on without micromanaging. If a tool could automatically generate action items

from our stand-up calls and assign them to the right people, it would save us hours every week.”

4.3 Functional Requirements

- FR1:** The system should allow users to create Workspaces and Projects with role-based access control (Admin, Manager, Member).
Priority: High
- FR2:** The Notes Module should support real-time simultaneous document editing with a rich text editor and slash commands.
Priority: High
- FR3:** The Meet Module should facilitate P2P audio/video conferencing and screen sharing among project members.
Priority: High
- FR4:** The system should automatically initialize a linked project note when a new meeting is started.
Priority: Medium
- FR5:** The system should support bidirectional synchronization, allowing note checklists to convert into Board tasks and vice versa.
Priority: High
- FR6:** The system should notify users when they are assigned to a task or mentioned in a workspace.
Priority: Medium
- FR7:** The Board Module should provide a Kanban board with customizable columns (e.g., To Do, In Progress, Done) for visual workflow management.
Priority: High
- FR8:** The system should allow users to assign task priorities (Low, Medium, High, Critical) and due dates to Board items.
Priority: Medium
- FR9:** The Meet Module should support in-call chat messaging for sharing links and text without interrupting the audio/video stream.
Priority: Medium
- FR10:** The system should provide a global search functionality to locate notes, tasks, and meeting logs across all workspaces.
Priority: Medium
- FR11:** The system should utilize the Gemini API to automatically extract and suggest actionable tasks from meeting notes.
Priority: Low
- FR12:** The system should utilize the Whisper API to generate searchable text transcriptions of recorded meetings.
Priority: Low

- FR13:** Users should be able to invite new members to a workspace via email or shareable invite links.
Priority: Medium
- FR14:** The system should maintain an activity log tracking edits, task assignments, and meeting history for audit purposes.
Priority: Low
- FR15:** The Notes Module should support file attachments and embedded media (images, PDFs) within documents.
Priority: Medium

4.4 Non-Functional Requirements

- NFR1:** Latency for real-time state synchronization (typing and board updates) should remain under 200ms.
Priority: High
- NFR2:** The platform services should be independent, and downtime of one service should not affect the others (fault isolation).
Priority: Medium
- NFR3:** The system must ensure secure data handling and authentication to protect sensitive workspace data.
Priority: High
- NFR4:** The backend architecture must be horizontally scalable to support an increasing number of concurrent WebRTC connections.
Priority: Medium
- NFR5:** User emails should be verified by sending a confirmation email containing a verification link to their personal mailbox.
Priority: High
- NFR6:** The user interface should remain responsive, with initial page load times under 3 seconds on standard broadband connections.
Priority: Medium
- NFR7:** API rate limiting should be implemented to prevent abuse, with a limit of 100 requests per minute per user token.
Priority: Medium
- NFR8:** The platform should support a minimum of 50 simultaneous users in a single workspace without performance degradation.
Priority: Medium
- NFR9:** All backend services must expose health-check endpoints for automated monitoring by Prometheus.
Priority: Low

NFR10: Structured logs must be emitted in JSON format using Pino, with log retention of at least 30 days in Elasticsearch.

Priority: Low

NFR11: The system should be deployable via Docker containers with a single command using Docker Compose.

Priority: Medium

NFR12: Load testing with k6 must demonstrate the ability to handle 100 concurrent Web-Socket connections without errors.

Priority: Medium

5 System Design

5.1 Client-Server Architecture

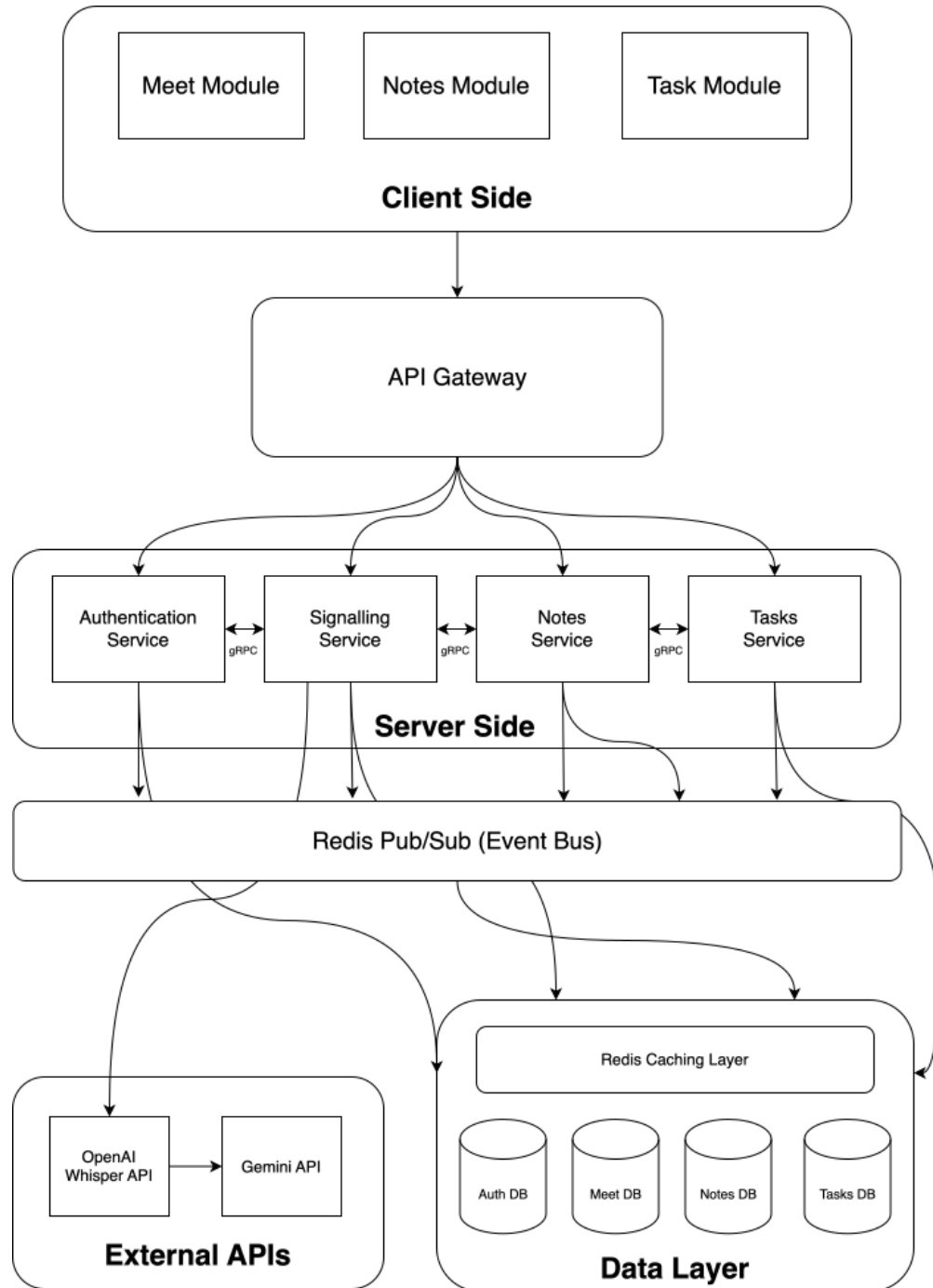


Figure 1: Client-Server Architecture diagram showing the interaction between frontend, backend services, and external APIs.

5.2 User Flow Chart

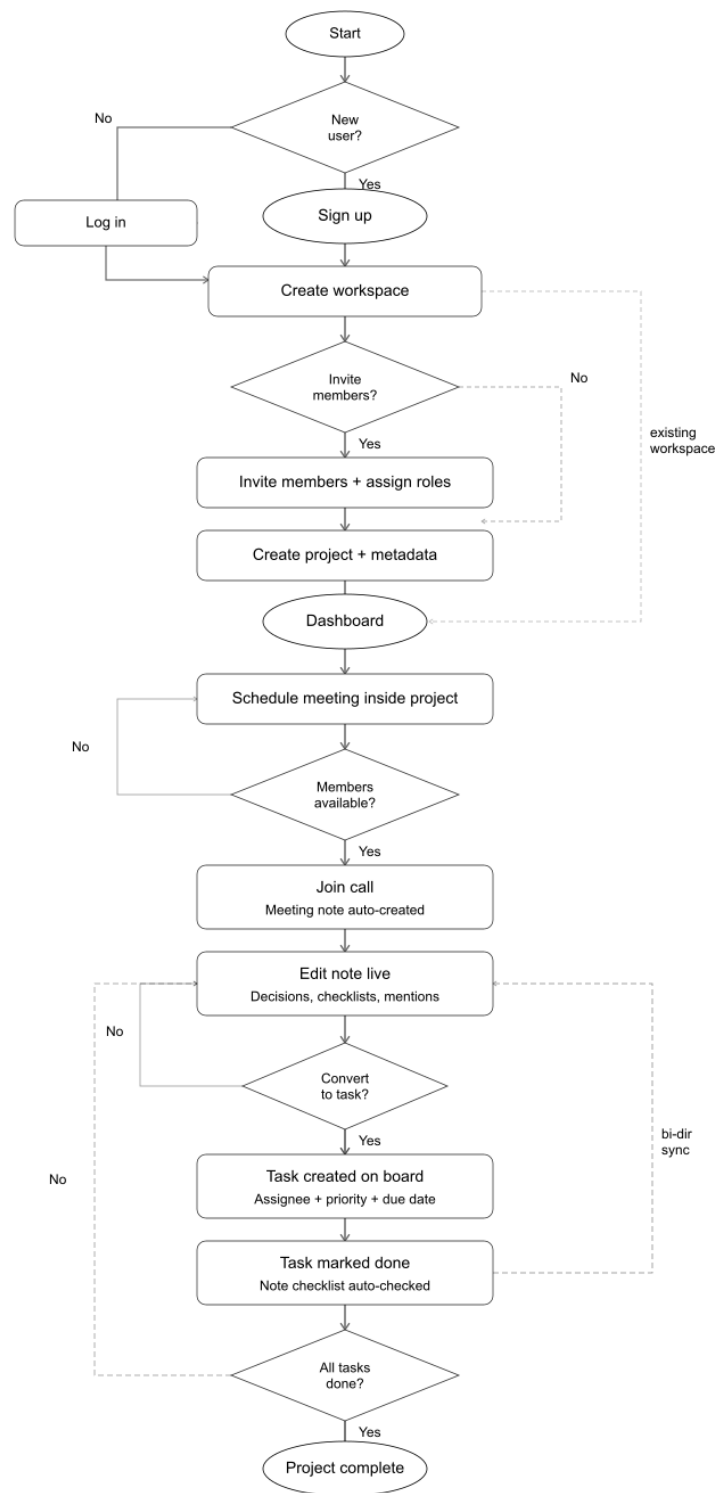


Figure 2: User Flow Chart illustrating the navigation paths and interactions within the Vero platform.

5.3 Workspace Separation

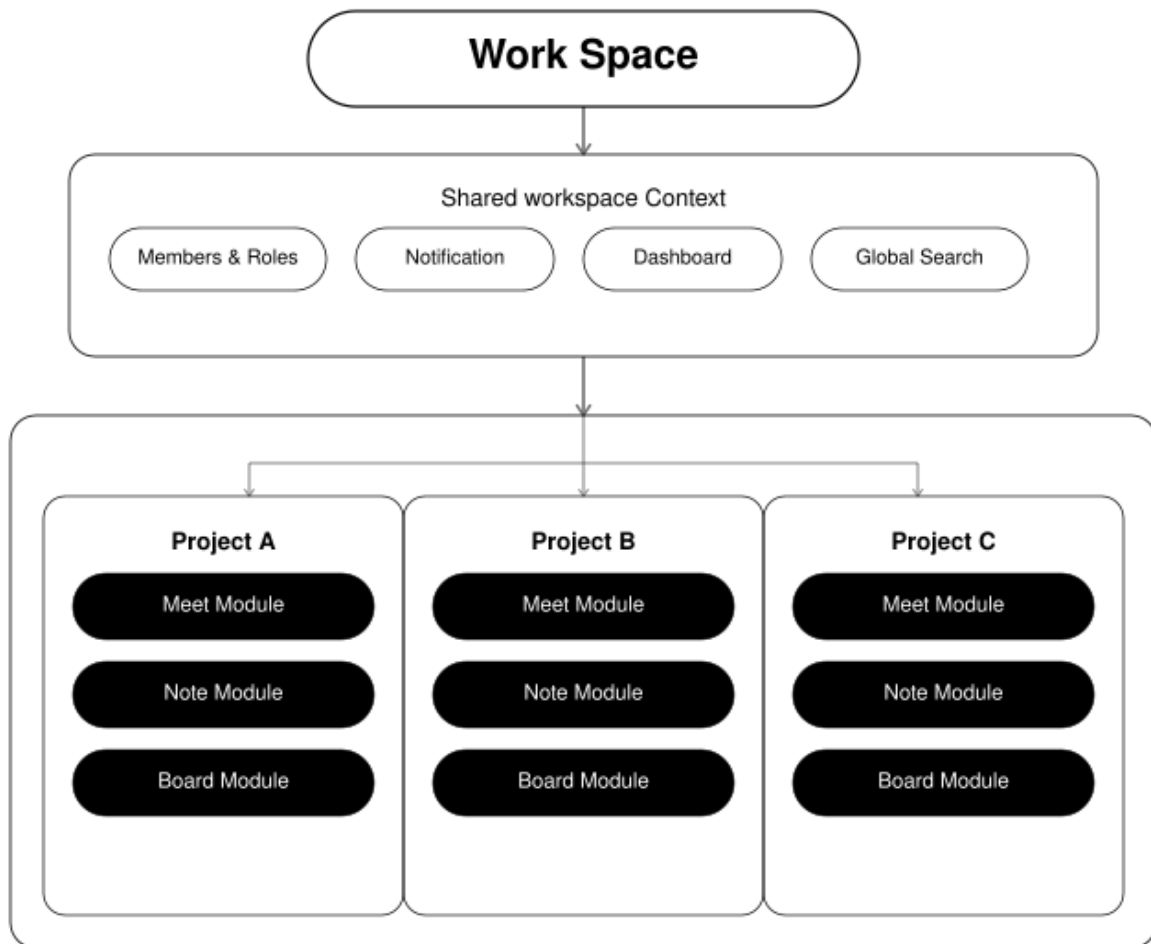


Figure 3: Workspace Separation diagram depicting the microservices architecture and isolation of different platform components.

6 Key Features

Vero integrates multiple productivity facets into a unified ecosystem. To support the **Shared Project Context**, the platform's functionalities are categorized into the following core areas:

6.1 Workspace & Organization

- **Centralized Dashboard:** A global view summarizing active projects, pending tasks, and recent activities.
- **Role-Based Access Control (RBAC):** Secure management of workspace members, assigning specific roles (Admin, Manager, Member) to control access.

- **Global Search:** A unified search index that instantly locates notes, tasks, or meeting logs throughout the workspace.

6.2 Meet Module (Communication)

- **P2P Audio & Video:** Low-latency peer-to-peer conferencing built directly into the project environment.
- **Screen Sharing:** High-fidelity screen sharing for technical demonstrations and presentations.
- **Auto-Initialized Context:** Initiating a call automatically generates a linked meeting document in the Notes Module.

6.3 Notes Module (Documentation)

- **Simultaneous Note Taking:** Real-time collaborative editing using CRDT synchronization to ensure seamless multi-user typing.
- **Rich Text Editor & Slash Commands:** A versatile block editor allowing users to quickly format text or insert dynamic components using "/" commands.
- **Embedded Components:** Native support for inserting interactive tables, calendars, and inline code blocks directly into documents.

6.4 Board Module (Task Management)

- **Kanban Board:** Visual workflow planning with customizable columns for tracking project progress.
- **Bidirectional Sync:** Checklists created in the Notes module can be instantly converted into trackable Kanban tasks, and vice versa.
- **Automated Notifications:** Smart alerts that notify users via the dashboard when they are assigned a task or mentioned in a note.

6.5 AI & Automation (Integrations)

- **Smart Task Extraction:** Utilizing the Gemini API to automatically parse meeting notes and suggest actionable tasks for the Board.
- **Meeting Transcription:** Leveraging the Whisper API to generate accurate, searchable text logs of voice conversations.

7 Tools and Technology

To support the real-time, highly coupled nature of the modules, **Vero** will be built using the following technology stack:

7.1 Frontend Application:

- **Frameworks:**
 - **React.js** – Core library for building the interactive web UI with a component-based architecture.
 - **Electron** – Wraps the web app into a cross-platform desktop application.
- **Languages:**
 - **TypeScript (JavaScript)** – Strongly typed superset of JavaScript for safer, more maintainable code.
- **Libraries:**
 - **Socket.io** – Enables real-time, bidirectional communication between the client and server over WebSockets.
 - **Yjs** – CRDT library enabling conflict-free simultaneous synchronization of the Block Editor.
 - **Zustand** – Lightweight global state management for the client.

7.2 Backend & Real-time Infrastructure:

- **Frameworks:**
 - **Node.js with Express** – Server-side runtime and framework for building scalable RESTful APIs.
- **Communication Protocols:**
 - **WebRTC** – Enables peer-to-peer audio and video calls directly between users.
 - **WebSockets** – Handles real-time synchronization of text and board state.
 - **gRPC** – High-performance RPC framework for efficient inter-service communication between backend microservices.
- **Message Broker:**
 - **Redis Pub-Sub** – Distributed event streaming platform for reliable, asynchronous message passing between services.
- **Database & Caching:**
 - **NeonDB** – Serverless Postgres database serving as the primary relational store.
 - **Redis** – In-memory cache for low-latency updates and session management.
- **Libraries:**

- **Better Auth** – Modern authentication library handling user sessions and security.
- **Simple Peer** – Simplifies WebRTC peer connection management for audio/video calls.
- **Pino** – Low-overhead structured logger for Node.js, optimized for high-throughput logging.

7.3 External APIs & Services:

- **AI Integration:**

- **Gemini API** – Powers intelligent workspace features like content generation and summarization.
- **Whisper API** – Provides voice-to-text transcription for meeting notes and captions.

7.4 DevOps & Observability:

- **Containerization & Orchestration:**

- **Docker** – Containerization for consistent development and deployment environments.

- **Monitoring & Metrics:**

- **Prometheus** – Time-series metrics collection and alerting for system health monitoring.
- **Grafana** – Visualization and dashboarding platform for real-time metrics and logs.

- **Testing:**

- **k6** – Open-source load testing tool for performance and stress testing of APIs and WebSocket connections.

- **Log Aggregation:**

- **Elasticsearch** – Centralized log ingestion and search engine for collecting and processing structured logs from all services.

- **Version Control:**

- **GitHub** – Version control and collaborative code hosting.

8 Proposed Timeline

The following Gantt chart outlines the development phases for Vero over a 14-week period.

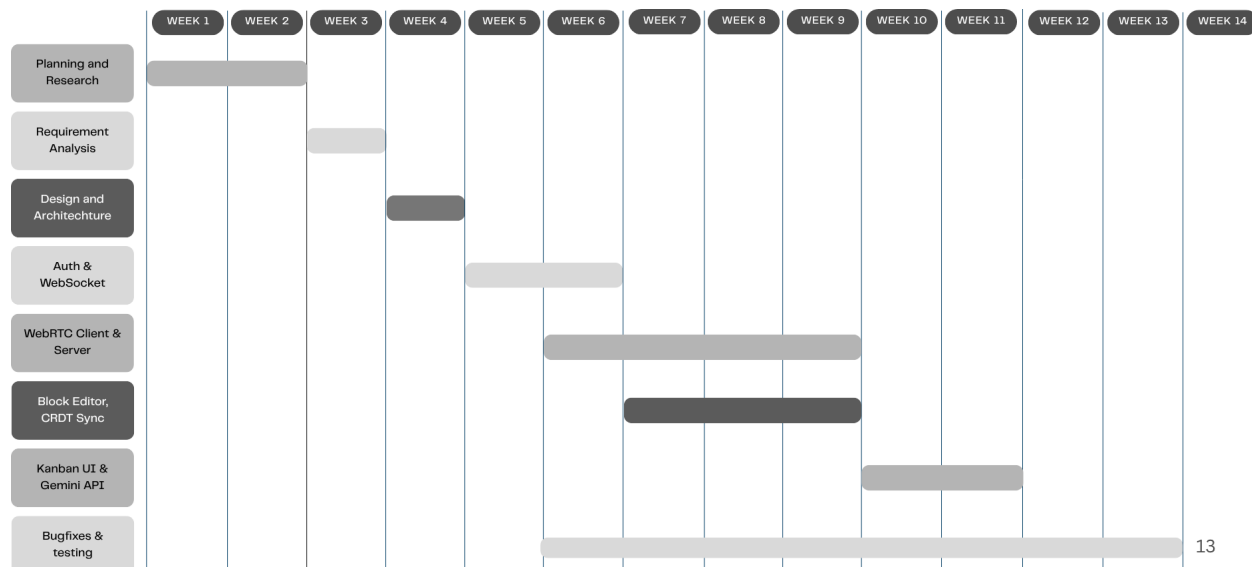


Figure 4: Project Roadmap and Development Timeline

9 Action Taken Based on Feedback

In this section, we outline the specific modifications and refinements made to the project proposal following the initial consultation with our supervisor.

Feedback 1: "The combined system is likely not to be as performant as individual applications for the three distinct modules. How will you maintain efficiency and retain the performance level of standalone apps?"

Action: By integrating a microservices architecture, we resolve potential performance bottlenecks. Additionally, by deploying multiple instances of individual services, we ensure high efficiency and effectively manage concurrent user requests.

Feedback 2: "Is the claim that a unified workspace improves productivity backed by any established research or empirical studies?"

Action: We have added a dedicated research section citing studies by Rubinstein, Meyer, and Evans (2001) on the "Tax on Productivity" and Mark, Gudith, and Klocke (2008) on the cognitive costs of interruptions. These references provide a scientific foundation for how Vero's unified environment reduces "switch costs" and workplace stress.

Feedback 3: "The platform must prioritize automation in note-taking and task assignment; without these features, it will fall short in comparison to established competitors."

Action: We have enhanced the 'AI & Automation' module by integrating the Gemini API for smart task extraction and the Whisper API for automated meeting transcription. These

additions ensure that the transition from a live meeting to an actionable task board is seamless and automated, providing a competitive edge over manual documentation tools.

Feedback 4: "The project should be packaged with only the necessary features, focusing strictly on the requirements of the target audience to avoid feature bloat."

Action: We conducted a stakeholder analysis to refine our feature set, ensuring that every core module – Meet, Notes, and Board – directly serves the specific needs of students, project managers, and software engineers. This focused approach ensures the application remains lightweight and high-performance while fulfilling its primary purpose.